Economics and reinforcement learning both study sequential decision-making under uncertainty, but they descend from different intellectual traditions. Economics is fundamentally an *inference culture*. Its central task is to understand the world. An economist's "model" is a specification of preferences, beliefs, constraints, and an equilibrium concept. The RL tradition is instead a *control culture*. Its central task is to act in the world. An RL researcher's "model" is a transition kernel $P(s'|s, a)$ and a reward function $r(s, a)$. These are different mathematical objects serving fundamentally different scientific purposes.

The two cultures maintain different relationships with data. Economists work primarily with observational data, and endogeneity is the central obstacle. Identification is its defining challenge which involves dealing with functional forms, equilibrium definitions, instrumental variables, regression discontinuities, and natural experiments. RL researchers have traditionally enjoyed what might be called simulator omnipotence. They often own the data-generating process and can create the variation in their data and fit any manner of nonparametric functions through millions of simulated samples. They face computational rather than statistical constraints. This difference shapes the cultures, from algorithm design to what counts as a valid result and what is the standard of evidence.

The two fields also differ in their treatment of human agency. To an economist, the human (or firm) is the optimizer, a rational or boundedly rational agent solving her own dynamic program, whose revealed choices are the data to be explained. To an RL researcher, the algorithm is the optimizer and the human is part of the environment. A customer's purchasing decision is not modeled as utility maximization but as a stochastic reward signal from which the algorithm learns. Even the word "learning" itself carries different meaning across the divide. In economic theory, learning means Bayesian updating of beliefs about unknown parameters. In reinforcement learning, learning means executing a recursive stochastic approximation algorithm to find the fixed point of the Bellman operator. The shared vocabulary with divergent meanings creates persistent confusion for researchers crossing from one field to the other. This chapter attempts to provide a systematic translation between the two cultures.

0.1 Core Distinctions

RL organizes its core problems along axes that do not map directly onto well known categories. *Prediction* refers to estimating $V^\pi(s)$ or $Q^\pi(s,a)$ for a fixed policy π (policy evaluation), not forecasting observable variables. *Control* refers to finding the policy π^* that maximizes expected discounted return (policy optimization), not the inclusion of regressors. Because prediction concerns evaluating a specific policy, a closely related question is whether the data was generated by the policy being studied. *On-policy* methods evaluate and improve the same policy that generates the data. *Off-policy* methods learn about a target policy π from data generated by a different behavioral policy μ. This distinction is central to causal inference (Section **??**), where off-policy evaluation is precisely counterfactual policy evaluation.

A separate question concerns how the algorithm accesses experience. *Online* RL learns while interacting with the environment, collecting new data as a consequence of its own actions. *Offline* RL (also called batch RL) learns exclusively from a fixed dataset of previously collected transitions, with no ability to gather additional samples. The offline setting is closer to standard empirical work, where the dataset is given and the analyst cannot run new experiments; the online setting corresponds more closely to adaptive experimental design or sequential decision problems. Note that "online" in RL carries no timing constraint; it means only that the agent generates fresh experience. *Real-time* RL, by contrast, imposes hard deadlines on the perception-action loop, as in robotics or mechatronics. Every real-time RL system is online, but most online RL (games, recommender systems) are not real-time.

Aside from data access is what the algorithm does with that data. In RL, the word *model* refers strictly to the environment's mathematical dynamics, the transition kernel $P(s'|s,a)$ and reward function $r(s,a)$. A *model-based* RL algorithm explicitly constructs or is given a mathematical representation of P and r, then computes a policy by planning through that representation (for example, using a simulator or the known rules of a game, as in AlphaZero). A *model-free* algorithm computes the value function or policy directly from experienced transitions without ever building an explicit representation of the transition probabilities. Crucially, "model-free" need not mean the algorithm lacks complete access to any model of the environment. A model-free algorithm could interact

with a simulator that internally implements a complete model of the environment; the distinction is that the algorithm never extracts or plans through the transition probabilities, treating the simulator as a black box that merely returns sample transitions. However, equally a "model-free" algorithm could also learn in-field where there is genuinely no access to a "model" of the environment but direct access to the environment itself (for reasons discussed later, this is rarely done). In economics, "model-based" connotes structural estimation, the specification of preferences, technology, beliefs, and an equilibrium concept. In RL, however, one can be "model-based" and plan through a given representation of P and r without making any structural economic assumptions about human or firm behavior. Conversely, a structural econometric model may internally use a "model-free" RL algorithm as a computational tool to find the fixed point of a Bellman equation, as in Section **??**.

Every RL system passes through two phases. In the *training phase*, the agent interacts with an environment (simulated or real) and updates its parameters. In the *execution phase* (also called the deployment phase), the policy is frozen and used to make decisions without further updates. This distinction is critical for interpreting "online." Online training in RL almost always takes place inside a simulator (and not in the "real world"). AlphaGo Zero trained online through millions of self-play games; when it faced Lee Sedol, its weights were frozen and it was purely executing its trained policy. Some deployed systems in Section **??** followed this pattern cleanly; DiDi's dispatch system trained a value function from historical trip data, then deployed with fixed weights. Others blur the boundary. The hotel revenue management system in Section **??** updated Q-values from realized returns after each completed episode during live operation, making it an *in-field* learner rather than a frozen executor. Bandit algorithms (Section **??**) also learn in-field by design, updating demand estimates from real customer responses during deployment. The term *in-field* is useful here; it denotes online learning where the environment is a live market with real economic consequences, as opposed to online learning inside a simulator where exploration is free, cheap and safe. One can use "operational phase", "in-field", '"live" interchangeably for this post-training regime where the system faces real and serious consequences, regardless of whether parameters continue to update.

A related terminological hazard concerns the word *inference*. In machine learning, "inference" refers to executing a frozen model, a forward pass producing outputs from in-

puts. In economics, "inference" means statistical inference, the construction of standard errors, confidence intervals, and hypothesis tests. This survey uses "inference" exclusively in the econometric sense and "execution" or "deployment" when referring to applying a trained model (whether in a computer or in field). Established terms such as "Bayesian inference" and "variational inference," which refer to inference about parameters or distributions, are used where appropriate. The key takeaway is that most RL convergence results and sample complexity guarantees refer to the training phase, and interpreting them as claims about deployed performance requires additional argument. Table 1 summarizes these distinctions.

Table 1: The reinforcement learning lifecycle grid. Most RL research operates in the top-left cell. When economists hear "online," they typically picture the bottom-left.

	Training (parameters updated)	Execution (parameters frozen)
Simulator	· AlphaGo Zero self-play · RL solver in structural estimation · Bandit calibration via simulation	· Policy benchmarks on synthetic environments
Historical data	· RLHF reward model from preference logs · Causal OPE from observational records · Logged-bandit policy learning	· Off-policy evaluation of a target policy
Live market ("in-field")	· Bandit pricing experiments · Hotel RM Q-learning from live episodes	· DiDi dispatch with frozen weights · AlphaGo vs. Lee Sedol

A typical economic RL pipeline moves through the grid sequentially, from pre-training on historical logs (middle-left) to refinement in a simulator (top-left) to deployment with frozen weights (bottom-right). Bandits illustrate this fluidity; even a bandit algorithm that will ultimately learn in-field is typically calibrated in simulation and tuned on historical logs before any live deployment, because in-field exploration incurs real financial cost. The systems that do operate in-field arrive with exploration parameters, initial policies, and demand priors shaped by extensive offline preparation.

The Tmall e-commerce pricing project of **?** (Section **??**) illustrates this migration concretely. The team pre-trained a DQN from logged specialist pricing decisions using DQfD (historical data, training), then ran offline evaluation of the candidate policy on held-out transaction logs before any live deployment (historical data, execution). The

evaluated policy was deployed for 15 to 30 day field experiments on live Tmall traffic, with the agent receiving reward and observation signals from the market environment (live market, execution and training). **?** note that no accurate simulator exists for e-commerce pricing, so the project skipped the simulator row entirely, jumping from historical pre-training directly to live deployment. Not every application traverses all six cells of Table 1, but the grid clarifies which cells a given project could be working on.

0.2 Overlapping Terminology

In RL the *environment* is a formal object encompassing everything outside the agent (the DGP, other agents, market clearing conditions), whereas in economics "environment" refers more loosely to market structure or institutional rules.[1]

The terms for what is observed and measured diverge just as sharply. A *return* in finance is the yield on an investment, typically expressed as a percentage gain; in RL, the return $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$ is the discounted sum of future rewards from time t onward, the random variable whose expectation defines the value function. The RL usage is closer to what economists call the "present discounted value" of a stream of payoffs. A single complete sequence of interactions from an initial state to termination, $(s_0, a_0, r_1, s_1, \ldots, s_T)$, is called an *episode* (synonymously, *trajectory* or *rollout*).[2] Where an econometrician speaks of the *outcome*, meaning the dependent variable Y in a regression, RL has no single analog: the reward r_t is the per-period outcome, the return G_t is the cumulative outcome, and the value function $V^\pi(s)$ is the expected cumulative outcome conditional on state.

Several terms can be misleading. *Bootstrapping* in econometrics refers to Efron's resampling method (**?**); in RL, it means updating a value estimate using another value estimate rather than a complete realized return, as when a TD algorithm uses the target $r_{t+1} + \gamma V(s_{t+1})$ that depends on the current, uncertain estimate $V(s_{t+1})$ (**?**).

Learning carries at least four distinct meanings in this survey. In economic theory (Bayesian learning, adaptive expectations), learning refers to agents forming and refining

[1] A source of confusion is *generative model*. In machine learning broadly, this means a model of the joint distribution $P(X, Y)$ or a synthetic data generator (GANs, diffusion models). In RL theory, a generative model is a simulator oracle that, given any (s, a), returns a sample $s' \sim P(\cdot|s, a)$ and reward $r(s, a)$; the usage implies random-access simulation, stronger than sequential online interaction but weaker than knowing P analytically.

[2] The closest econometric analogs are a single panel unit's time series, a realization of a stochastic process, or one "history" in a dynamic model.

beliefs about unknown parameters of their environment. In supervised machine learning, learning means statistical estimation, fitting the weights of a parameterized model to minimize a loss function over data. In reinforcement learning, "learning" is primarily computation. When an RL agent "learns" a Q-function, it is executing a recursive stochastic approximation algorithm to find the fixed point of the Bellman operator. We say the algorithm is "learning" because it improves its policy iteratively through simulated or real experience, but mathematically it is solving a fixed-point problem. In many applications throughout this survey, particularly Section **??**, the RL algorithm is simply a numerical method for solving the Bellman equation; no actual human-like learning from experience is taking place. Finally, while RL draws inspiration from animal psychology (Section **??**), it is a drastic simplification of biological learning. Tabula rasa RL algorithms require millions of iterations of trial and error to discover policies that animals acquire rapidly. Real-world animal learning relies on innate priors, basic physical knowledge, and parental nurturing and should be distinct from RL-style "learning".

Function approximation in RL refers to representing value functions or policies using parameterized function classes (linear combinations of basis functions, kernel methods, or neural networks), which econometricians will recognize as sieve estimation or nonparametric series estimation, the approximation of an unknown function by projection onto a finite-dimensional basis.

The term *bandit* itself carries different mathematical content across disciplines. The classical multi-armed bandit in statistics and economics (**???**) is a Bayesian sequential allocation problem. The state is the agent's posterior belief over the unknown arm distributions, and the solution is the Gittins index, an optimal allocation rule derived from the theory of optimal stopping. In the RL and computer science literature, bandits are instead framed as frequentist regret-minimization problems. Algorithms such as UCB provide worst-case bounds on cumulative regret $\sum_{t=1}^{T}(\mu^* - \mu_{A_t})$ without requiring Bayesian priors. The two traditions ask fundamentally different questions, Bayesian optimality of the full sequential problem versus minimax regret rates over adversarial or stochastic environments, and their answers are not directly comparable. Section **??** adopts the regret framework because it connects more naturally to the sample complexity concerns that arise in economic field experiments.

The term *contextual bandit* is especially liable to misreading by economists. To an

economist, the "context" is simply the state variable x_t, and a contextual bandit looks like an MDP with an unknown reward parameter. What the RL literature signals by "context" is a specific structural restriction on transitions, the agent's action has no causal effect on the next context, so that $P(x_{t+1} \mid x_t, a_t) = P(x_{t+1})$. This exogeneity assumption separates the exploration problem (learning which arm is best given the current context) from the planning problem (choosing actions that influence future states). When contexts evolve exogenously, there is no long-horizon credit assignment, and the problem reduces to repeated one-period optimization under uncertainty. The term "contextual" therefore flags a modeling assumption about dynamics, not merely the presence of observable covariates.

A concrete example clarifies the gap. The RL literature frames recommender systems as contextual bandits, where user covariates are the context, the recommendation is the arm, and a click or rating is the reward. An economist might instead view movie recommendation as a two-sided learning problem in which the platform learns user preferences while users simultaneously explore the catalog and update their own tastes. The bandit formulation absorbs the user's utility maximization into the environment's reward signal and treats user arrivals as exogenous. This is a modeling choice, not a fact about the application. The object called a "bandit" in this formulation, a one-step decision under exogenous context, is a different mathematical object than the Bayesian sequential allocation problem of **?**, even though both carry the same name and are related.[3]

0.3 Structural Equivalences

Beyond terminological differences, several formal objects in RL and economics are mathematically identical. The softmax (or Boltzmann) policy used throughout RL is the multinomial logit model of **?**. The RL softmax policy selects actions according to

$$\pi(a \mid s) = \frac{\exp(Q(s,a)/\tau)}{\sum_{a' \in \mathcal{A}} \exp(Q(s,a')/\tau)}, \tag{1}$$

where $\tau > 0$ is a temperature parameter. In the discrete choice framework, $Q(s,a)$ plays the role of the deterministic component of utility $v(a \mid x)$, and τ is the scale parameter

[3] **?** proves that the Gittins index with a flat Gaussian prior achieves finite-time regret of the same order as UCB, and that the index decomposes as posterior mean plus an exploration bonus that shrinks toward zero near the horizon, structurally resembling but differing from the UCB confidence bound.

of the Type I extreme value (Gumbel) taste shocks ε_a. As $\tau \to 0$, the policy converges to the greedy (deterministic) policy, just as the logit choice probability concentrates on the utility-maximizing alternative as the variance of taste shocks vanishes.

The entropy regularization commonly added to RL objectives is the inclusive value (or log-sum-exp) from the discrete choice literature. The soft value function

$$V^{\text{soft}}(s) = \tau \log \sum_{a \in \mathcal{A}} \exp(Q(s,a)/\tau) \tag{2}$$

is identical to the McFadden surplus function $W(x) = \tau \log \sum_a \exp(v(a|x)/\tau) + C$, where C is Euler's constant. In the structural estimation literature, this object appears as the Emax function in dynamic discrete choice models following **?**.

The advantage function $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$ measures how much better action a is compared to the average action under π. In the discrete choice framework, this is precisely the choice-specific value function net of the ex-ante value function, a quantity that appears in the **?** CCP estimator for dynamic discrete choice models.

0.4 Notation

Table 2 maps the notation used throughout this survey to the most common econometric equivalents.

Table 2: Notation mapping between reinforcement learning and economics.

RL Term	Symbol	Economics Equivalent	Symbol
State	$s \in \mathcal{S}$	State variable, covariate	x_t, Ω_t
Action	$a \in \mathcal{A}$	Choice, control variable	d_t, u_t
Reward	$r(s,a)$	Per-period utility, payoff	$u(x,d)$
Discount factor	γ	Discount factor	β
Policy	$\pi(a\|s)$	Decision rule, CCP	$P(d\|x)$
Value function	$V^\pi(s)$	Ex-ante value function	$\bar{V}_\theta(x)$
Q-function	$Q^\pi(s,a)$	Choice-specific value function	$v_\theta(x,d)$
Return	G_t	Present discounted value	$\sum_{k=0}^{\infty} \beta^k u_{t+k}$
Transition	$P(s'\|s,a)$	State transition law	$f(x_{t+1}\|x_t, d_t)$
Learning rate	α	Step size	α_n
TD error	δ_t	Bellman residual at sample	–

References

Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.

John C. Gittins. Bandit processes and dynamic allocation indices. *Journal of the Royal Statistical Society: Series B (Methodological)*, 41(2):148–164, 1979.

V. Joseph Hotz and Robert A. Miller. Conditional choice probabilities and the estimation of dynamic models. *The Review of Economic Studies*, 60(3):497–529, 1993.

Tor Lattimore. Regret analysis of the finite-horizon Gittins index strategy for multi-armed bandits. In *Proceedings of the 29th Conference on Learning Theory (COLT)*, pages 1214–1245, 2016.

Jiaxi Liu, Xiaoqing Wang, Yuming Deng, Xingyu Wu, and Yidong Zhang. Dynamic pricing on E-commerce platform with deep reinforcement learning: A field experiment. *arXiv preprint arXiv:1912.02572*, 2019.

Daniel McFadden. Conditional logit analysis of qualitative choice behavior. *Frontiers in Econometrics*, pages 105–142, 1974.

Michael Rothschild. A two-armed bandit theory of market pricing. *Journal of Economic Theory*, 9(2):185–202, 1974.

John Rust. Optimal replacement of gmc bus engines: An empirical model of harold zurcher. *Econometrica*, 55(5):999–1033, 1987.

Richard S. Sutton. Learning to predict by the methods of temporal differences. *Machine Learning*, 3(1):9–44, 1988. doi: 10.1023/A:1022633531479.

William R. Thompson. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, 25(3–4):285–294, 1933.